

Sort and Merge

```
DATA DIVISION.
FILE SECTION.
FD StudentFile.
01 StudentDetails.
    02 StudentId          PIC 9(7) .
    02 StudentName.
        03 Surname        PIC X(3) .
        03 Initials       PIC XX.
    02 DateOfBirth.
        03 YOBirth        PIC 9(2) .
        03 MOBirth        PIC 9(2) .
        03 DOBirth        PIC 9(2) .
    02 CourseCode        PIC X(4) .
    02 Grant              PIC 9(4) .
    02 Gender             PIC X.
```

The StudentFile is a sequential file sequenced upon ascending StudentId.

Write a program to display the number of students taking each course. How?



Simplified Sort Syntax.

SORT **WorkFileName** { **ON** { **ASCENDING**
DESCENDING } **KEY** {SortKeyIdentifier} ... } ...
USING {InFileName} ...
GIVING {OutFileName} ...

The **WorkFileName** identifies a temporary work file that the SORT process uses for the sort. It is defined in the **FILE SECTION** using an **SD** entry.

Each **SortKeyIdentifier** identifies a field in the record of the work file upon which the file will be sequenced.

When more than one **SortKeyIdentifier** is specified, the keys decrease in significance from left to right (leftmost key is most significant, rightmost is least significant).

InFileName and **OutFileName**, are the names of the input and output files. These files are automatically opened by the SORT. When the SORT executes they must **not** be already open.

Sort Example.

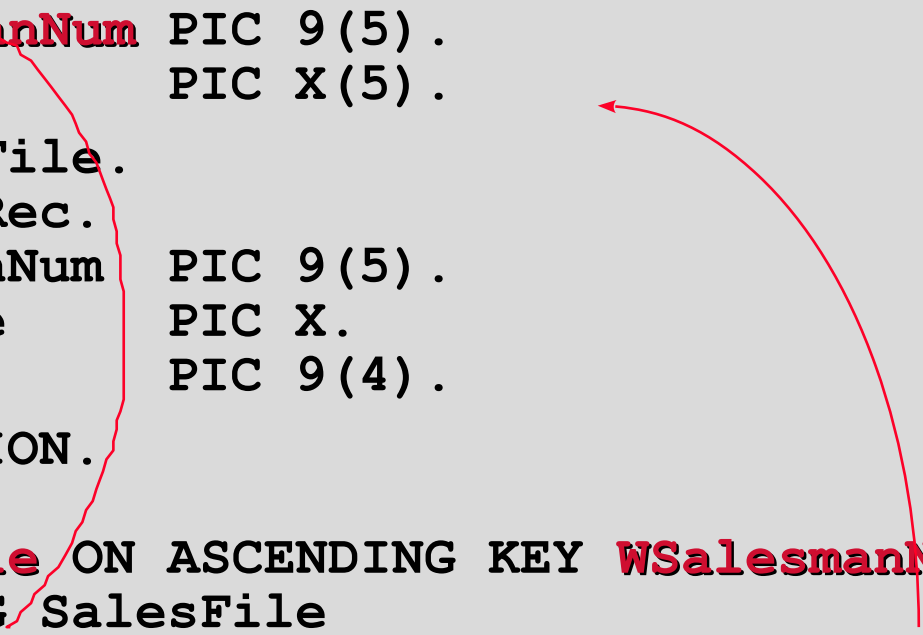
```
FD  SalesFile.
01  SalesRec.
    02  FILLER          PIC X(10) .

SD  WorkFile.
01  WorkRec.
    02  WSalesmanNum PIC 9(5) .
    02  FILLER          PIC X(5) .

FD  SortedSalesFile.
01  SortedSalesRec.
    02  SalesmanNum     PIC 9(5) .
    02  ItemType        PIC X.
    02  QtySold         PIC 9(4) .

PROCEDURE DIVISION.
Begin.
    SORT WorkFile ON ASCENDING KEY WSalesmanNum
        USING SalesFile
        GIVING SortedSalesFile.

    OPEN INPUT SortedSalesFile.
```



A red curved arrow originates from the **WorkFile** entry in the SORT statement and points to the **SortedSalesFile** entry, indicating the data flow from the input file to the output file.

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.

 SELECT WorkFile ASSIGN TO "WORK.TMP".

WorkFile.

01 WorkRecord.

02 ProvinceCode PIC 9.

02 SalesmanCode PIC 9(5) .

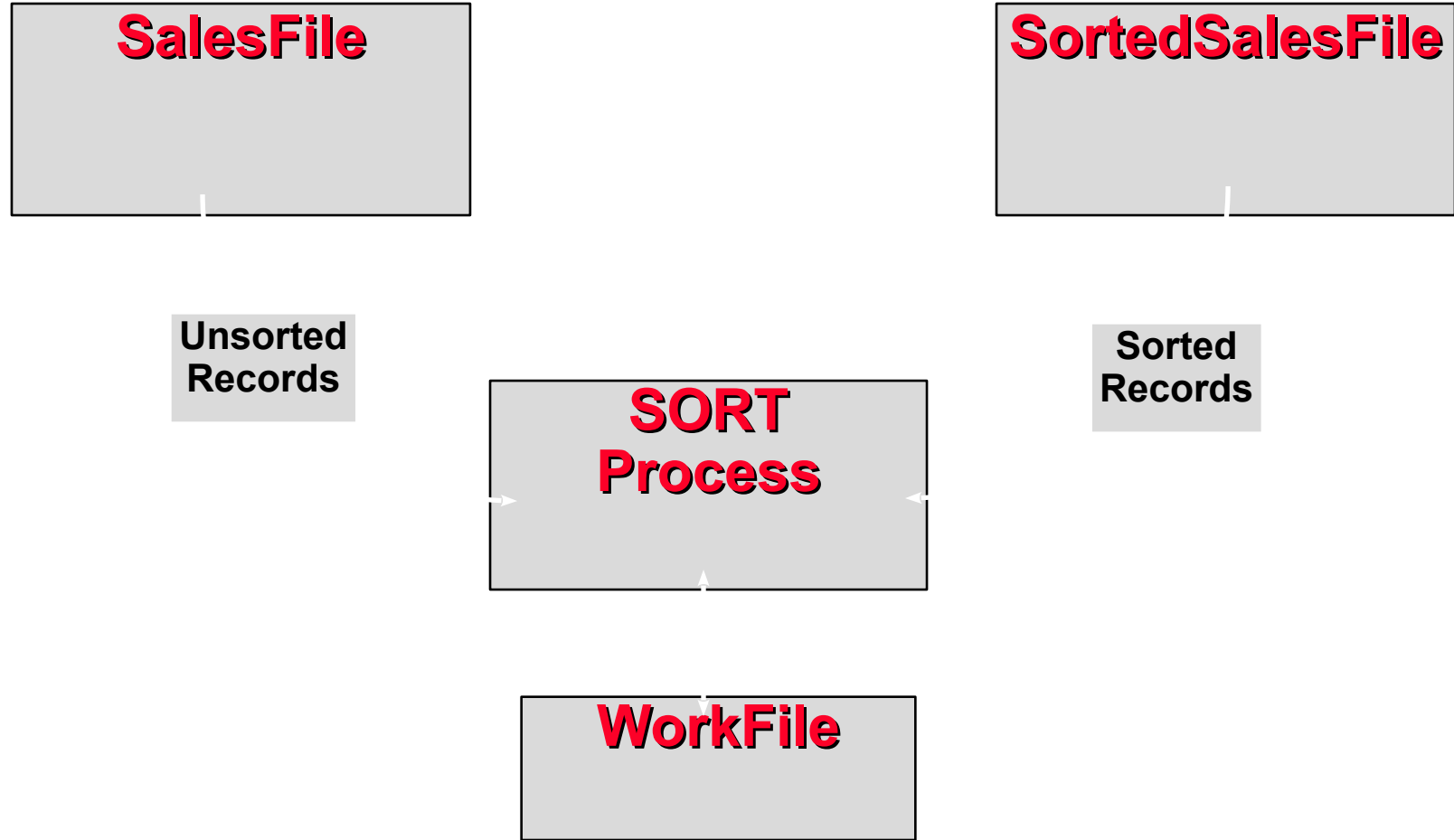
02 FILLER PIC X(19) .

Begin.

 SORT WorkFile ON ASCENDING KEY ProvinceCode
 DESCENDING KEY SalesmanCode
 USING UnsortedSales
 GIVING SortedSales.

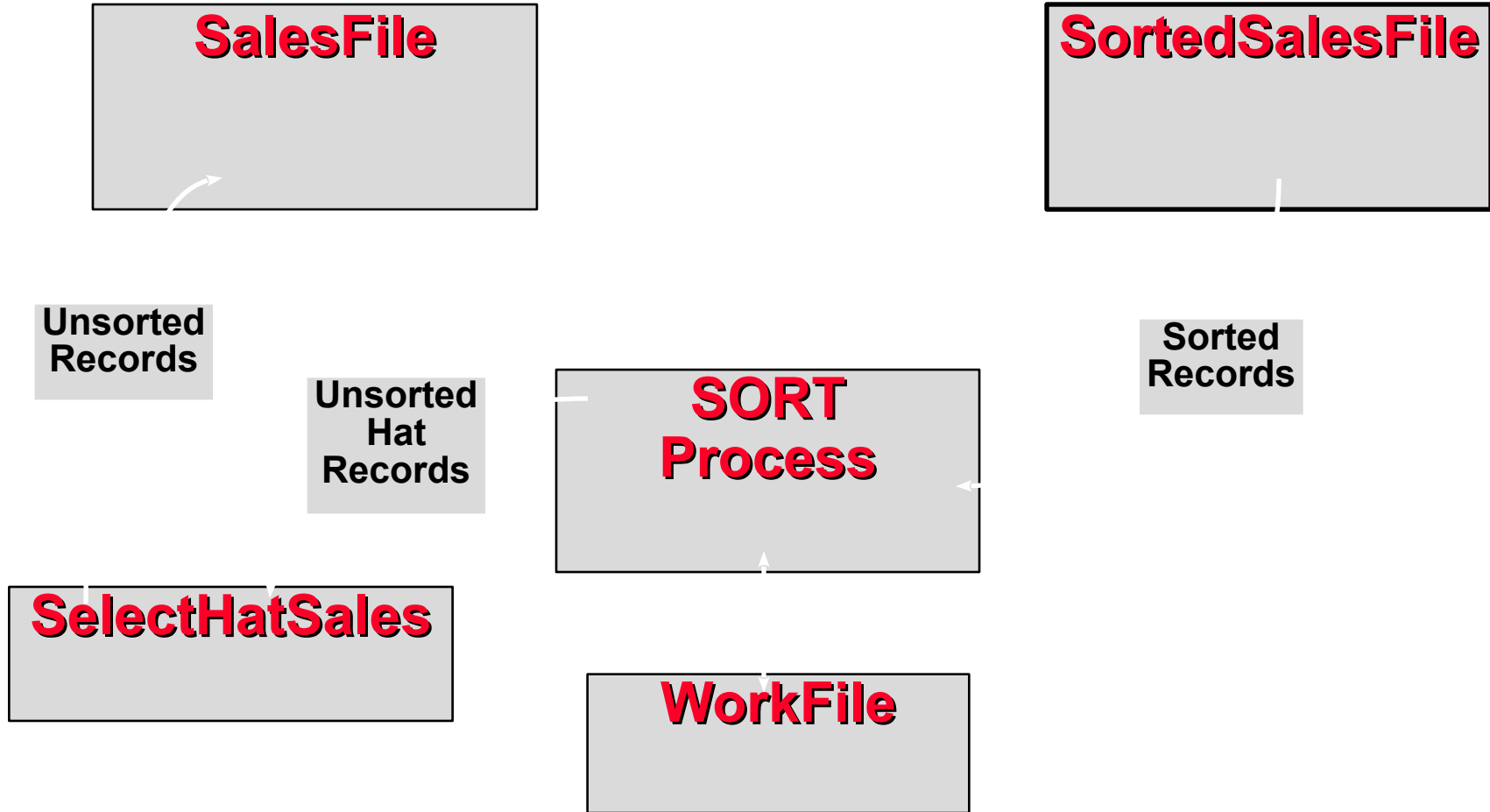
OPEN INPUT SortedSales.

How the SORT works.



`SORT WorkFile ON ASCENDING KEY WSalesmanNum
USING SalesFile
GIVING SortedSalesFile.`

How the INPUT PROCEDURE works.



`SORT WorkFile ON ASCENDING KEY WSalesmanNum`
`INPUT PROCEDURE IS SelectHatSales`
`GIVING SortedSalesFile.`

INPUT PROCEDURE Template

```
OPEN INPUT InFileName
READ InFileName RECORD
PERFORM UNTIL Condition
    RELEASE SDWorkRec
    READ InFileName RECORD
END-PERFORM
CLOSE InFile
```



INPUT PROCEDURE - Example

```
FD SalesFile.
01 SalesRec.
   88 EndOfSales          VALUE HIGH-VALUES.
   02 FILLER              PIC 9(5) .
   02 FILLER              PIC X.
      88 HatRecord        VALUE "H" .
   02 FILLER              PIC X(4) .

SD WorkFile.
01 WorkRec.
   02 WSalesmanNum        PIC 9(5) .
   02 FILLER              PIC X(5) .

FD SortedSalesFile.
01 SortedSalesRec.
   02 SalesmanNum         PIC 9(5) .
   02 ItemType            PIC X.
   02 QtySold             PIC 9(4) .

PROCEDURE DIVISION.
Begin.
    SORT WorkFile ON ASCENDING KEY WSalesmanNum
                INPUT PROCEDURE IS SelectHatSales
                GIVING SortedSalesFile.
```

New Version

```
SelectHatSales.  
  OPEN INPUT SalesFile  
  
  READ SalesFile  
    AT END SET EndOfSales TO TRUE  
  END-READ  
  PERFORM UNTIL EndOfSales  
    IF HatRecord  
      RELEASE WorkRec FROM SalesRec  
    END-IF  
    READ SalesFile  
      AT END SET EndOfSales TO TRUE  
    END-READ  
  END-PERFORM  
  
CLOSE SalesFile.
```

Old Version

```
SelectHatSales SECTION.  
BeginHatSales.  
    OPEN INPUT SalesFile  
    READ SalesFile  
        AT END SET EndOfSales TO TRUE  
    END-READ  
    PERFORM GetHatSales UNTIL EndOfSales  
    CLOSE SalesFile  
    GO TO SelectHatSalesExit.  
  
GetHatSales.  
    IF HatRecord  
        RELEASE WorkRec FROM SalesRec  
    END-IF  
    READ SalesFile  
        AT END SET EndOfSales TO TRUE  
    END-READ.  
  
SelectHatSalesExit  
    EXIT.
```

ENVIRONMENT DIVISION.
INPUT-OUTPUT SECTION.
FILE-CONTROL.

SELECT WorkFile ASSIGN TO "WORK.TMP".

SD WorkFile.
01 WorkRecord.
88 EndOfWorkFile VALUE HIGH-VALUES.
02 ProvinceCode PIC 9.
88 ProvinceIsUlster VALUE 4.
02 SalesmanCode PIC 9(5).
02 FILLER PIC X(19).
FD UnsortedSales.
01 FILLER PIC X(25).
FD SortedSales.
01 SortedRec.
88 EndOfSalesFile VALUE HIGH-VALUES.
02 ProvinceCode PIC 9.
02 SalesmanCode PIC 9(5).
02 ItemCode PIC 9(7).
02 ItemCost PIC 9(3)V99.
02 QtySold PIC 9(7).

PROCEDURE DIVISION.

Begin.

 SORT WorkFile

 ON ASCENDING KEY ProvinceCode

 SalesmanCode

 INPUT PROCEDURE IS SelectUlsterRecs

 GIVING SortedSales.

OPEN INPUT SortedSales.

 OPEN INPUT UnsortedSales

 READ UnsortedSales INTO WorkRec

 AT END SET EndOfSalesFile TO TRUE

 END-READ

PERFORM UNTIL EndOfSalesFile

 IF ProvinceIsUlster RELEASE WorkRec

 END-IF

 READ UnsortedSales INTO WorkRec

 AT END SET EndOfSalesFile TO TRUE

 END-READ

END-PERFORM

CLOSE UnsortedSales

Full Sort Syntax.

SORT WorkFileName { **ON** { **ASCENDING**
DESCENDING } **KEY** {SortKeyIdentifier} ... } ...

[**WITH** **DUPLICATES** **IN ORDER**]

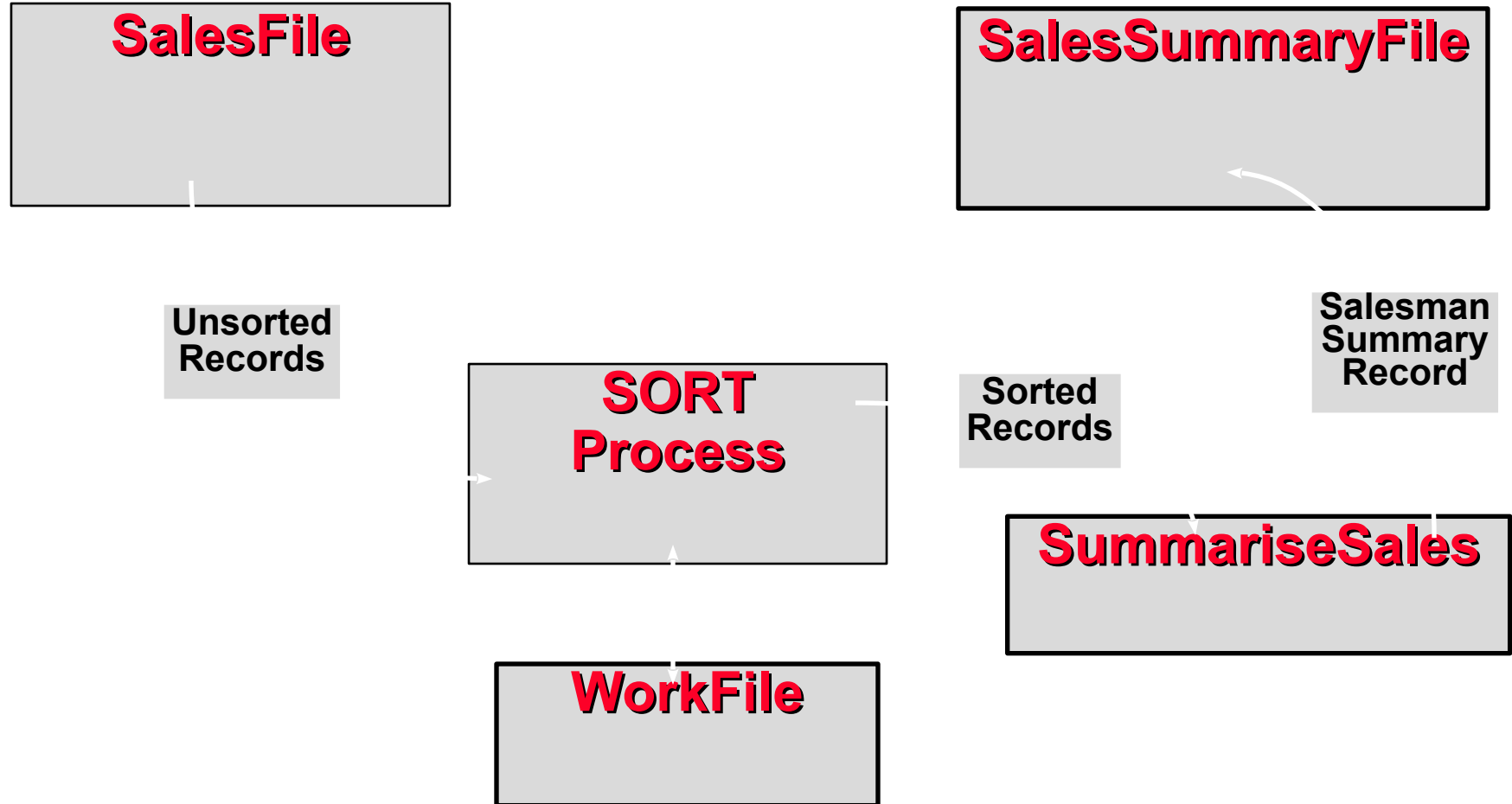
[**COLLATING** **SEQUENCE** **IS** AlphabetName]

{ **INPUT PROCEDURE** **IS** ProcedureName [{ **THRU**
THROUGH } ProcedureName]
USING {InFileName} ... }

{ **OUTPUT PROCEDURE** **IS** ProcedureName [{ **THRU**
THROUGH } ProcedureName]
GIVING {OutFileName} ... }

ProcedureName is the name of a section or paragraph.

How the OUTPUT PROCEDURE works.



`SORT WorkFile ON ASCENDING KEY WSalesmanNum
USING SalesFile
OUTPUT PROCEDURE IS SummariseSales.`

OUTPUT PROCEDURE Template

```
OPEN OUTPUT OutFile
RETURN SDWorkFile RECORD
PERFORM UNTIL Condition
    WRITE OutRec
    RETURN SDWorkFile RECORD
END-PERFORM
CLOSE OutFile
```


Output PROCEDURE - Example

```
FD   SalesFile.
01   SalesRec          PIC X(10) .
SD   WorkFile.
01   WorkRec.
      88   EndOfWorkFile    VALUE HIGH-VALUES.
      02   WSalesmanNum PIC 9(5) .
      02   FILLER          PIC X.
      02   WQtySold         PIC X(4) .
FD   SalesSummaryFile.
01   SummaryRec.
      02   SalesmanNum    PIC 9(5) .
      02   TotalQtySold  PIC 9(6) .
PROCEDURE DIVISION.
Begin.
      SORT WorkFile ON ASCENDING KEY WSalesmanNum
              USING SalesFile
              OUTPUT PROCEDURE IS SummariseSales.
      OPEN INPUT SalesSummaryFile.
      PERFORM PrintSummaryReport.
```

SummariseSales.

OPEN OUTPUT SalesSummaryFile

RETURN WorkFile

AT END SET EndOfWorkFile TO TRUE

END-RETURN

PERFORM UNTIL EndOfWorkFile

MOVE WSalesmanNum TO SalesmanNum

MOVE ZEROS TO TotalQtySold

PERFORM UNTIL WSalesManNum NOT = SalesmanNum

OR EndOfWorkFile

ADD WQtySold TO TotalQtySold

RETURN WorkFile

AT END SET EndOfWorkFile TO TRUE

END-RETURN

END-PERFORM

WRITE SummaryRec

END-PERFORM

CLOSE SalesSummaryFile.

RELEASE and RETURN Syntax

Write

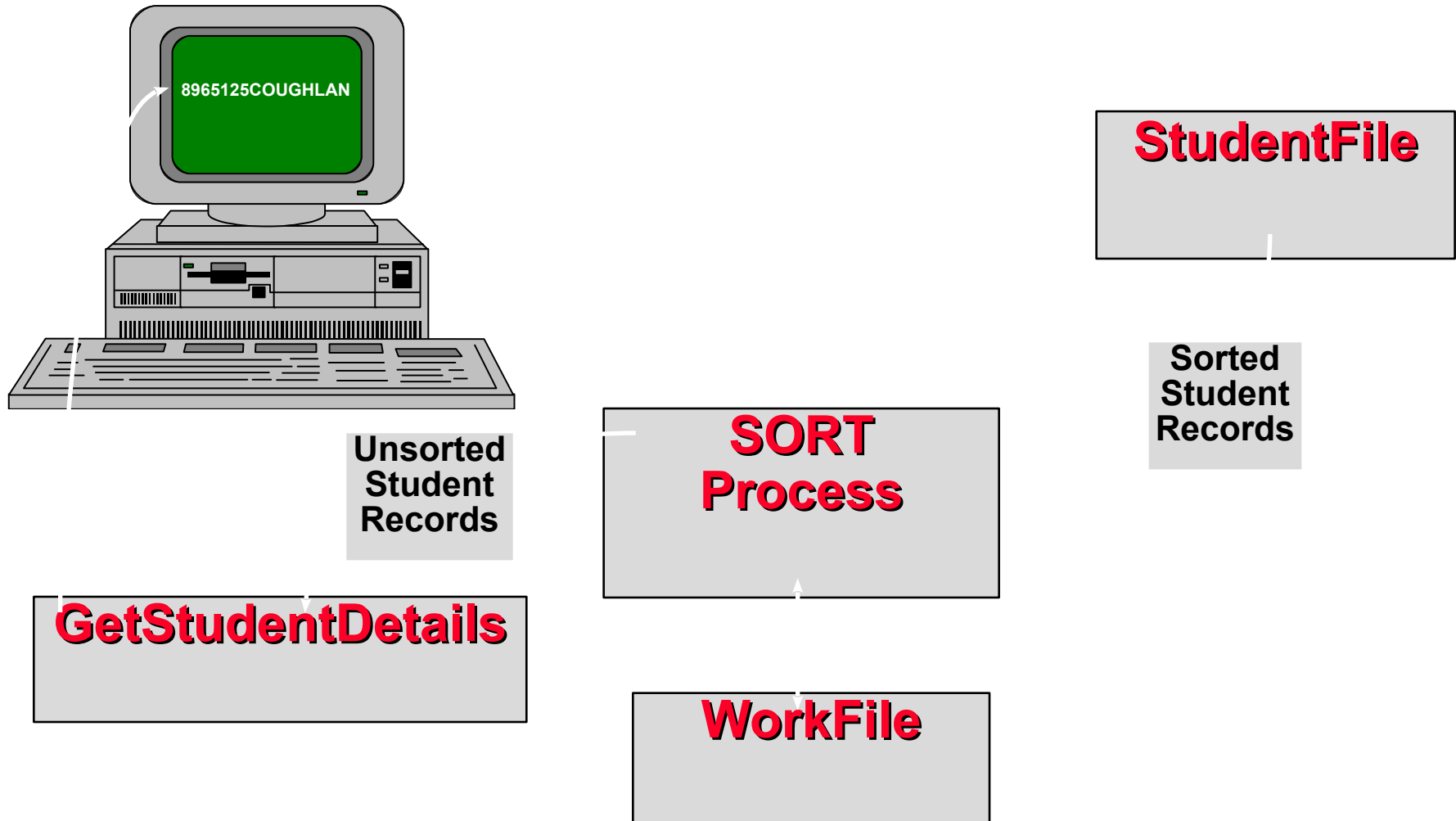
```
RELEASE SDFRecordName [FROM Identifier]
```

Read

```
RETURN SDFFileName RECORD [INTO Identifier]  
    AT END StatementBlock  
END-RETURN
```



Feeding the SORT from the keyboard.



`SORT WorkFile ON ASCENDING KEY WStudentId
INPUT PROCEDURE IS GetStudentDetails
GIVING StudentFile.`

ENVIRONMENT DIVISION.

INPUT-OUTPUT SECTION.

FILE-CONTROL.

 SELECT StudentFile ASSIGN TO "SORTSTUD.DAT"

 ORGANIZATION IS LINE SEQUENTIAL.

 SELECT WorkFile ASSIGN TO "WORK.TMP".

DATA DIVISION.

FILE SECTION.

FD StudentFile.

01 StudentDetails PIC X(32).

SD WorkFile.

01 WorkRec.

 02 WStudentId PIC 9(7).

 02 FILLER PIC X(25).

PROCEDURE DIVISION.

Begin.

 SORT WorkFile ON ASCENDING KEY WStudentId

 INPUT PROCEDURE IS GetStudentDetails

 GIVING StudentFile.

 STOP RUN.

GetStudentDetails.

 DISPLAY "Enter student details using template below."

 DISPLAY "Enter no data to end."

 DISPLAY "NNNNNNNNSSSSSSSSSSIIYYMMDDCCCCGGGGS".

 ACCEPT WorkRec.

 PERFORM UNTIL WorkRec = SPACES

 RELEASE WorkRec

 ACCEPT WorkRec

 END-PERFORM.

MERGE Syntax.

MERGE SDFFileName { ON { ASCENDING
DESCENDING } KEY {SortKeyIdentifier} ... } ...
[COLLATING SEQUENCE IS AlphabetName]
USING FDFFileName { FDFFileName } ...
{ OUTPUT PROCEDURE IS ProcedureName [{ THRU
THROUGH } ProcedureName] }
GIVING { FDFFileName } ...

The Merge takes two or more identically sequenced files and combines them, according to the key values specified, to produce a combined file which is then output to an output file or OUTPUT PROCEDURE.

e.g.

MERGE WorkFile ON ASCENDING KEY StudentId
USING InsertionsFile, StudentFile
GIVING NewStudentFile.



ENVIRONMENT DIVISION.

INPUT-OUTPUT SECTION.

FILE-CONTROL.

```
        SELECT StudentFile      ASSIGN TO "STUDENTS.DAT"
              ORGANIZATION IS LINE SEQUENTIAL.
        SELECT InsertionsFile ASSIGN TO "TRANSINS.DAT"
              ORGANIZATION IS LINE SEQUENTIAL.
        SELECT NewStudentFile ASSIGN TO "STUDENTS.NEW"
              ORGANIZATION IS LINE SEQUENTIAL.
        SELECT WorkFile          ASSIGN TO "WORK.TMP".
```

DATA DIVISION.

FILE SECTION.

FD StudentFile.

01 StudentRec PIC X(32) .

FD InsertionsFile.

01 InsertionRec PIC X(32) .

FD NewStudentFile.

01 NewStudentRec PIC X(32) .

SD WorkFile.

01 WorkRec.

 02 WStudentId PIC 9(7) .

 02 FILLER PIC X(25) .

PROCEDURE DIVISION.

Begin.

```
        MERGE WorkFile ON ASCENDING KEY WStudentId
              USING InsertionsFile, StudentFile
              GIVING NewStudentFile.
```

STOP RUN.